

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud-Native Automated Deployment

Data 2026/5/11 Class AI 1

Name Weijie Xu ID 202383910029

一、 Experimental purpose

(1) To Understand the Basic Concepts of Cloud Computing

The first purpose of this experiment is to help students understand the basic ideas of cloud computing through practical operation. Instead of only learning cloud computing from theoretical definitions, students are required to complete a real deployment task on GitHub. Through this process, students can understand that cloud computing provides computing resources, storage resources, and deployment services through the Internet. The experiment also helps students realize that users do not need to manage physical servers directly. They can use cloud platforms to complete project hosting, automatic building, and online access. This provides a direct understanding of cloud service models and cloud-based application deployment.

(2) To Learn the Workflow of GitHub-Based Project Hosting

Another important purpose of this experiment is to help students become familiar with GitHub as a cloud-based code hosting platform. By logging in to GitHub, locating the given repository, and forking it into their own account, students can understand how a project can be copied, stored, and managed in a cloud environment. The fork operation allows each student to create an independent version of the original project, which is similar to creating a private instance in the cloud ecosystem. Through this step, students can also understand the basic workflow of repository management, including browsing files, editing source code, committing changes, and tracking project updates.

(3) To Master the Basic Use of GitHub Pages as a PaaS Service

This experiment also aims to help students understand how GitHub Pages works as a Platform-as-a-Service service. By enabling GitHub Pages and selecting GitHub Actions as the deployment source, students can experience a managed hosting process. In this process, students only need to prepare and modify the website content, while

the platform is responsible for building, deploying, and providing access to the website. This helps students understand the main feature of PaaS: developers can focus on application content and logic without directly maintaining the underlying operating system, server software, network configuration, or deployment environment.

(4) To Understand Cloud Automation Through GitHub Actions

A key purpose of this experiment is to help students understand cloud automation. After modifying the index.html file and committing the changes, GitHub Actions automatically starts the Cloud-Deployment-Pipeline. This process allows students to observe how a cloud platform responds to code changes and automatically executes a deployment workflow. Through this step, students can understand the concept of on-demand computing, because GitHub creates a temporary virtual machine environment to process the build and deployment task. This helps students build a practical understanding of automation, workflow execution, and cloud-based computing resource scheduling.

(5) To Experience Continuous Deployment and Global Web Access

This experiment is designed to help students experience the basic process of continuous deployment. Once the GitHub Actions workflow finishes successfully, the website can be accessed through a live URL provided by GitHub Pages. Students can open the deployed website on their own laptop and verify whether the content has been updated correctly. This process shows that after each valid code change and commit, the cloud platform can automatically rebuild and redeploy the website. Through this operation, students can understand how continuous deployment improves development efficiency and how cloud platforms support global access to web applications.

(6) To Improve Practical Ability and Deployment Verification

The final purpose of this experiment is to improve students' hands-on ability in simple web content modification and deployment verification. Students are required to change their name in the HTML file and may further modify the theme color, font size, background style, or add multimedia elements such as images. These operations help students understand the relationship between source code changes and final web page presentation. At the same time, students need to take screenshots at different stages to prove that each step has been completed on their own computer. This requirement trains students to record experimental procedures clearly, verify

deployment results carefully, and reflect on problems encountered during the experiment.

二、Experiment Contents

- This experiment first requires students to log in to GitHub and obtain the experimental project resources from the given repository. By forking the original repository into their own GitHub account, students create an independent copy of the project in the cloud environment. This step allows students to understand how cloud-based code hosting works and how a shared project can be copied, managed, and modified individually through GitHub.
- Students need to enter the repository settings and enable GitHub Pages for website hosting. In the GitHub Pages configuration, GitHub Actions is selected as the deployment source. This operation changes the deployment method from manual uploading to an automated managed pipeline. Through this step, students can understand how a PaaS platform provides hosting services and how users can deploy a simple website without configuring a physical server.
- After committing the changes, students need to open the Actions tab and observe the Cloud-Deployment-Pipeline. GitHub Actions automatically starts the deployment workflow and uses a temporary cloud-based virtual machine to process the task. By observing the workflow status, students can understand how cloud automation works and how an online platform automatically builds and deploys a project after source code changes.
- When the GitHub Actions workflow finishes successfully, students should return to the GitHub Pages settings page and find the live website URL. Then, they need to open the URL on their own computer to check whether the website has been deployed successfully and whether the modified content is displayed correctly. This step verifies the complete deployment result and helps students understand global web access through cloud hosting.
- Students may further personalize the web page by modifying HTML and CSS elements. For example, they can change the background color, adjust the font style, modify the page layout, or add an image through an HTML image tag. After each modification is committed, GitHub Actions will automatically trigger a new deployment. This allows students to experience continuous deployment and understand how cloud platforms update different types of web content automatically.
- During the experiment, students need to record the operation process with screenshots. The screenshots should clearly show the key stages, including repository

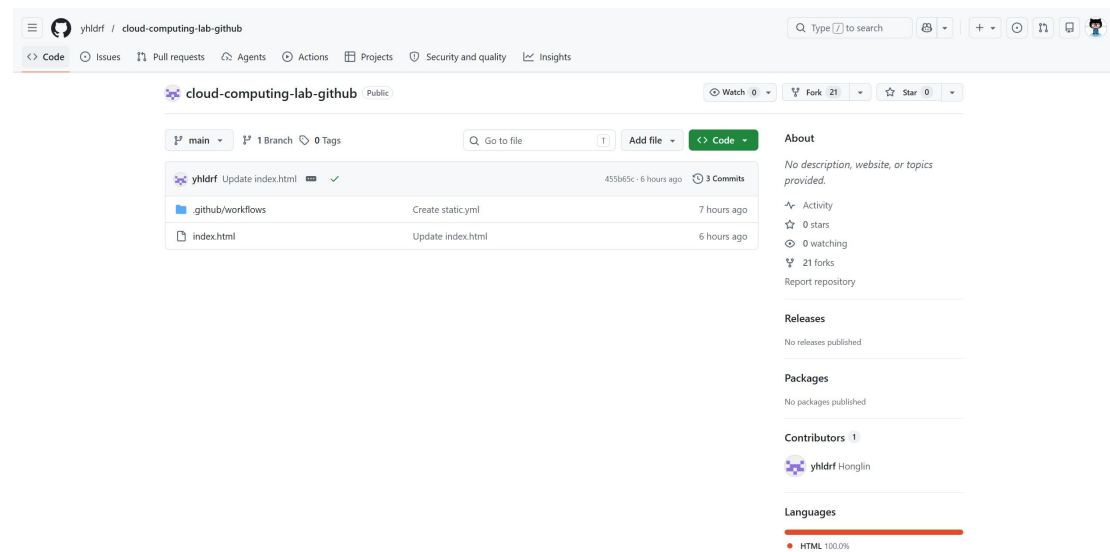
forking, GitHub Pages configuration, file modification, GitHub Actions workflow execution, and final website access. These screenshots should also contain identifiable information from the student's own computer, such as system time or desktop environment, so that the experimental process can be verified.

- At the end of the experiment, students need to reflect on the deployment process and answer several related questions, such as where the website is stored, whether any local software installation is required, and what problems were encountered during the experiment. This part helps students summarize the practical meaning of cloud deployment, identify possible operation errors, and propose reasonable solutions based on their own experimental experience.

三、 Detailed steps of the experiment

Step 1: GitHub Login and Repository Access

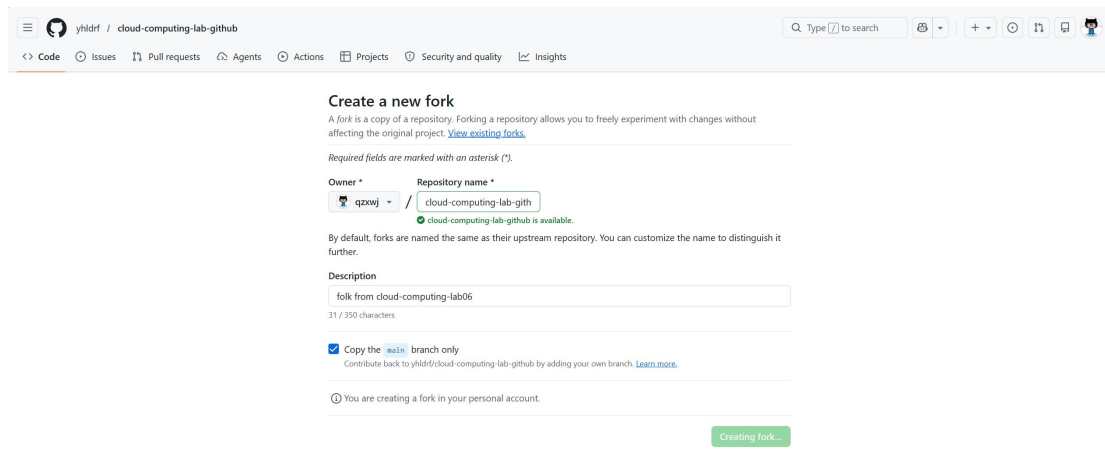
In this step, I logged in to GitHub and opened the experimental repository at <https://github.com/yhldrf/cloud-computing-lab-github>. This screenshot records the starting point of the experiment, where the required source code and deployment files are stored on a cloud-based code hosting platform. Instead of receiving the project as a local compressed file, I accessed it directly through GitHub, which provides repository storage, version control, web editing, and collaboration services. From the perspective of cloud computing, this step shows that project resources can be centrally stored and accessed through the Internet, allowing users to obtain the same experimental environment from different devices.



Step 2: Forking the Repository

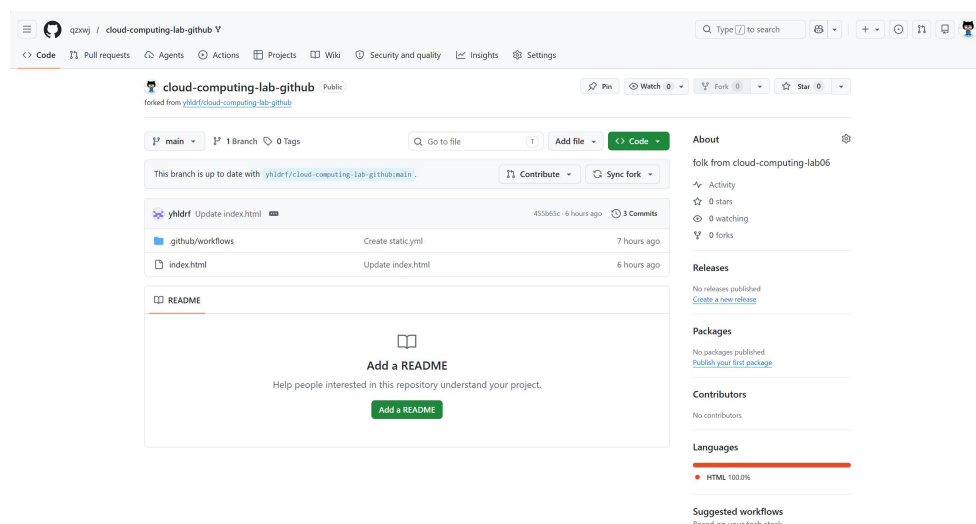
After entering the original repository page, I clicked the Fork button to create a copy

of the project under my own GitHub account. Forking does not simply download the files to my laptop; it creates an independent cloud-side repository that I can modify, configure, and deploy without changing the original project. This operation represents resource acquisition in the cloud ecosystem. The original repository works as a shared template, while the forked repository becomes my personal instance of that resource. This also shows the elasticity and convenience of cloud platforms, because a new working copy can be created quickly through the web interface.



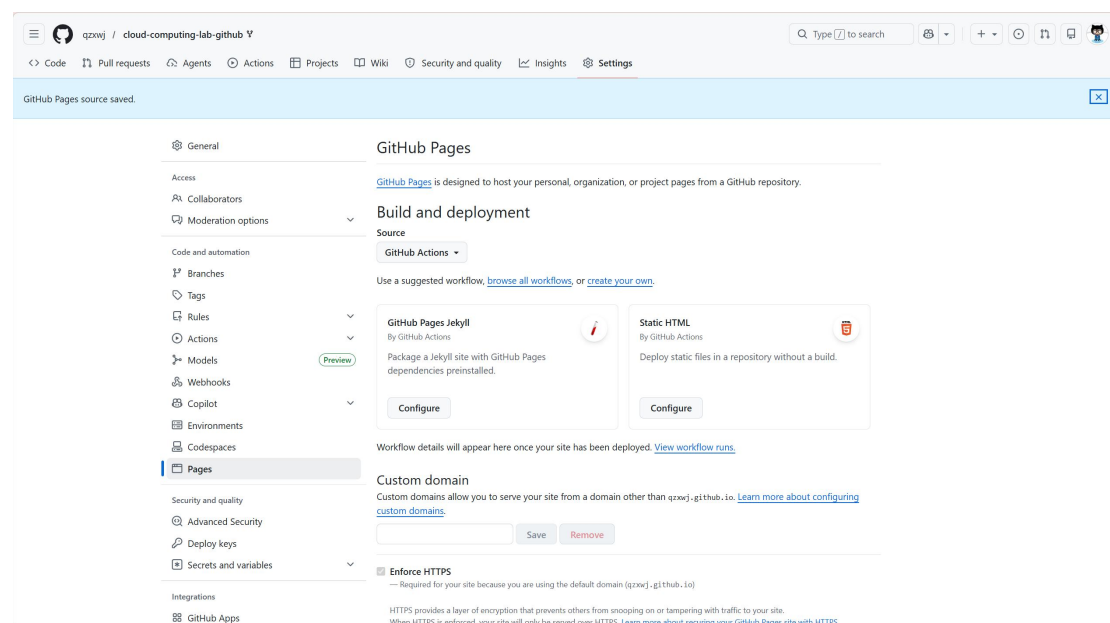
Step 3: Confirming the Forked Repository

This screenshot shows that the repository has been successfully forked into my own GitHub account. After the fork is created, I can see the same project files, but the repository owner has changed to my account. This means I now have permission to edit files, adjust repository settings, and trigger deployments independently. The cloud concept reflected here is isolated cloud workspace creation. Each user can build a separate copy from a common source, and the cloud platform manages the storage, metadata, access control, and version history for that copy.



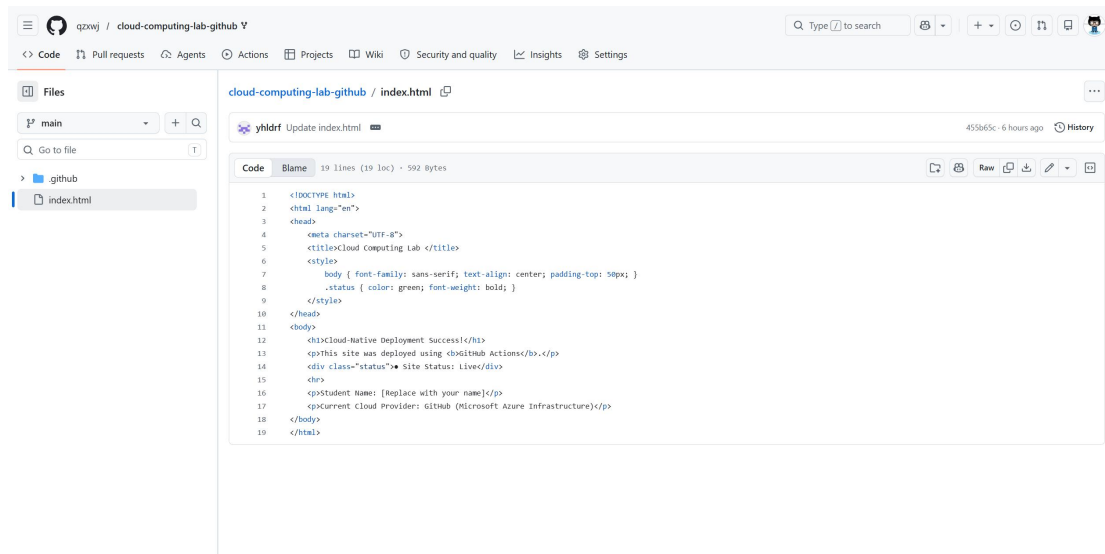
Step 4: Opening Repository Settings

Next, I entered the Settings page of the forked repository to configure the hosting service. The Settings page contains repository-level service options, including GitHub Pages, Actions permissions, branch information, and deployment-related controls. This step is important because the project must be connected to a managed hosting service before it can become a public website. At this stage, the repository is no longer only used for code storage. It is being prepared as a deployable cloud application, where the platform provides configuration panels instead of requiring manual server installation.



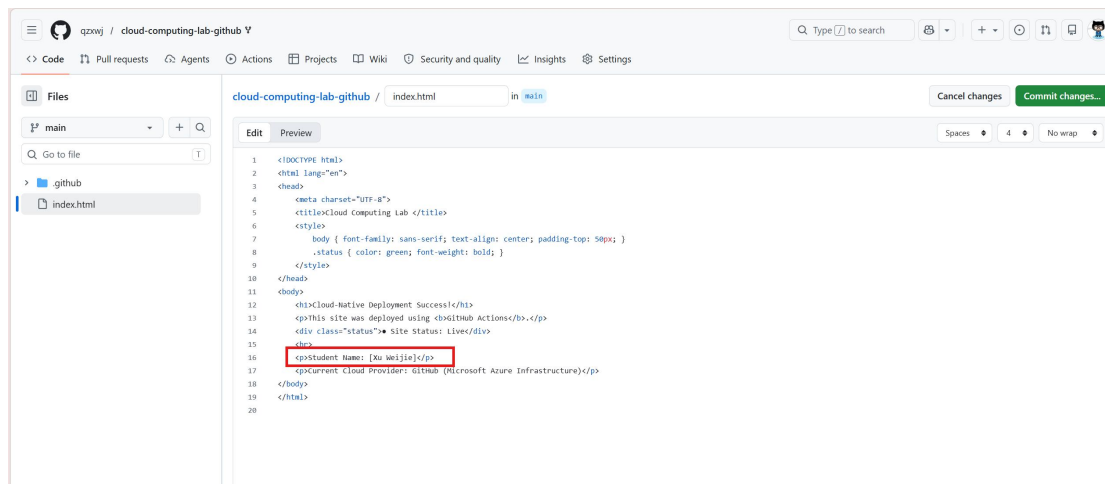
Step 5: Enabling GitHub Pages with GitHub Actions

In the Pages settings, I selected GitHub Actions as the build and deployment source. This means that GitHub Pages will not rely on a manual upload process. Instead, it will use a managed workflow to build and publish the site whenever the repository changes. This step demonstrates the idea of a managed pipeline service. The user only chooses the deployment source and commits the project files, while GitHub manages the workflow execution environment, deployment process, and hosting target. It reduces the need to maintain a separate web server and makes the deployment process more standardized.



Step 6: Opening index.html

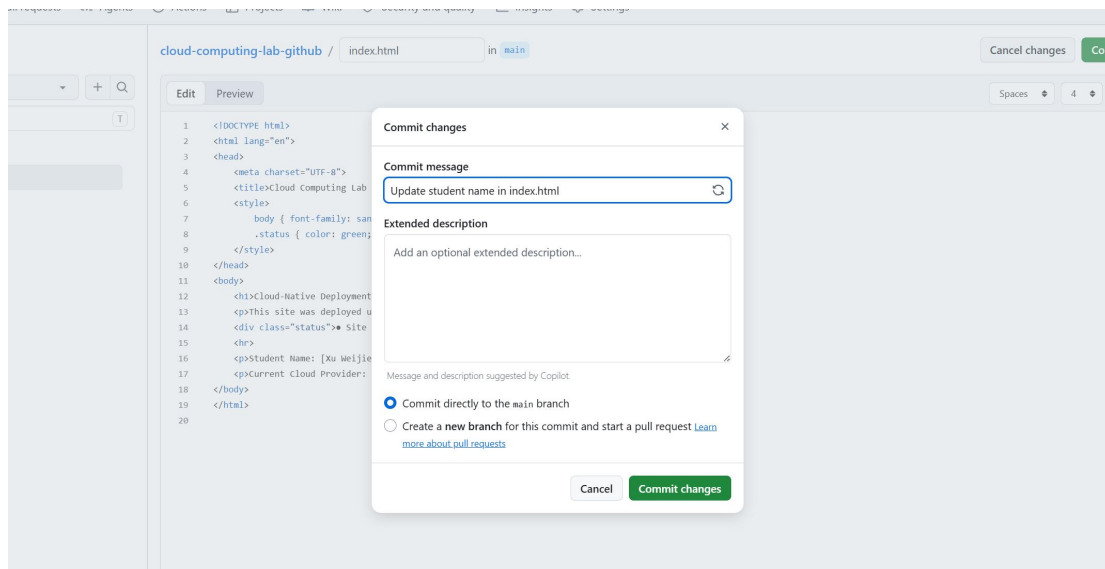
After enabling the hosting service, I opened the index.html file in GitHub's browser editor. The index.html file is the entry page of the static website, so changes made in this file will directly affect the content displayed after deployment. Using the browser editor also shows that basic cloud development can be completed without installing a local IDE or configuring a local runtime. The file is edited, stored, and versioned in the cloud repository, and every later commit can become an input for the deployment pipeline.



Step 7: Editing the Displayed Name

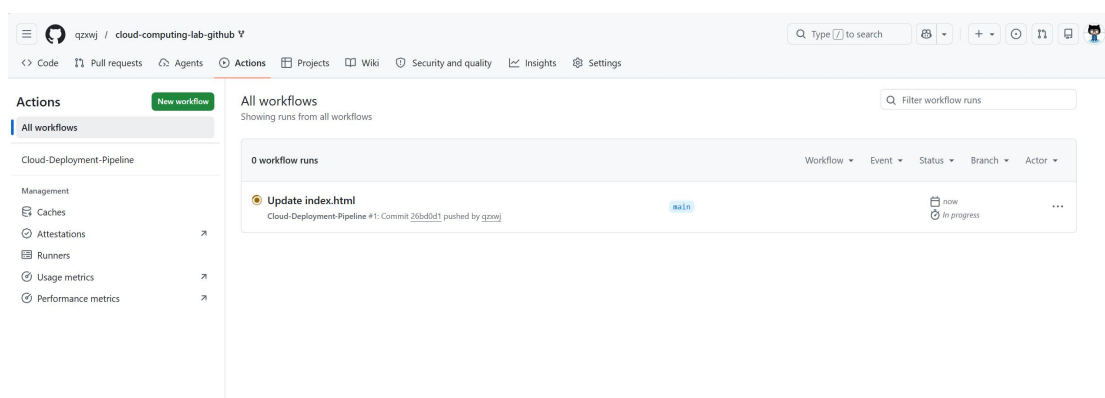
In the index.html file, I replaced [Enter Your Name Here] with my actual name. This is a simple content personalization step, but it is also a clear test case for deployment verification because the result can be observed directly on the final web page. By modifying visible text, I created a difference between the original project and my

forked version. From a cloud deployment perspective, this change proves that user-level content updates can be stored in the repository and then automatically delivered to the hosted website after the pipeline runs.



Step 8: Committing Changes

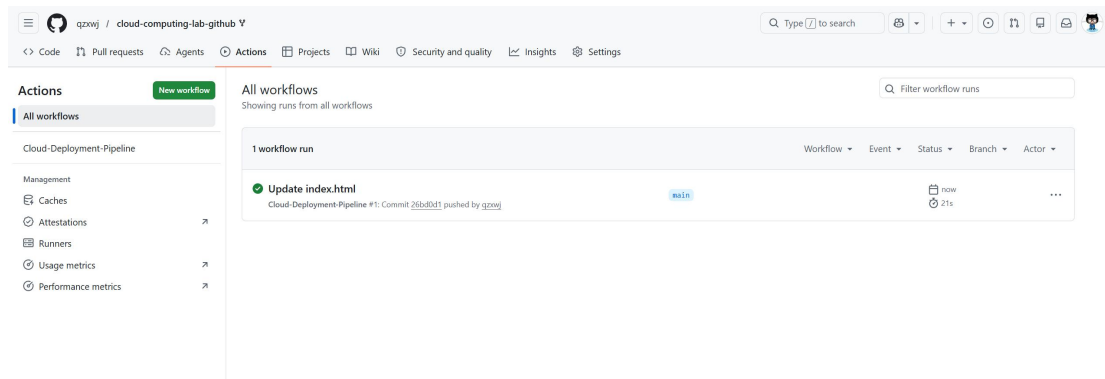
After editing the file, I clicked Commit changes to save the modification into the forked repository. A commit records the updated file content, the time of change, and the version history of the project. This step is essential because GitHub Actions uses commits as workflow triggers. Once the commit is created, the repository state changes from the previous version to a new version, and the cloud automation system can detect that change. In this way, version control and automated deployment are connected through the commit operation.



Step 9: Opening the Actions Tab

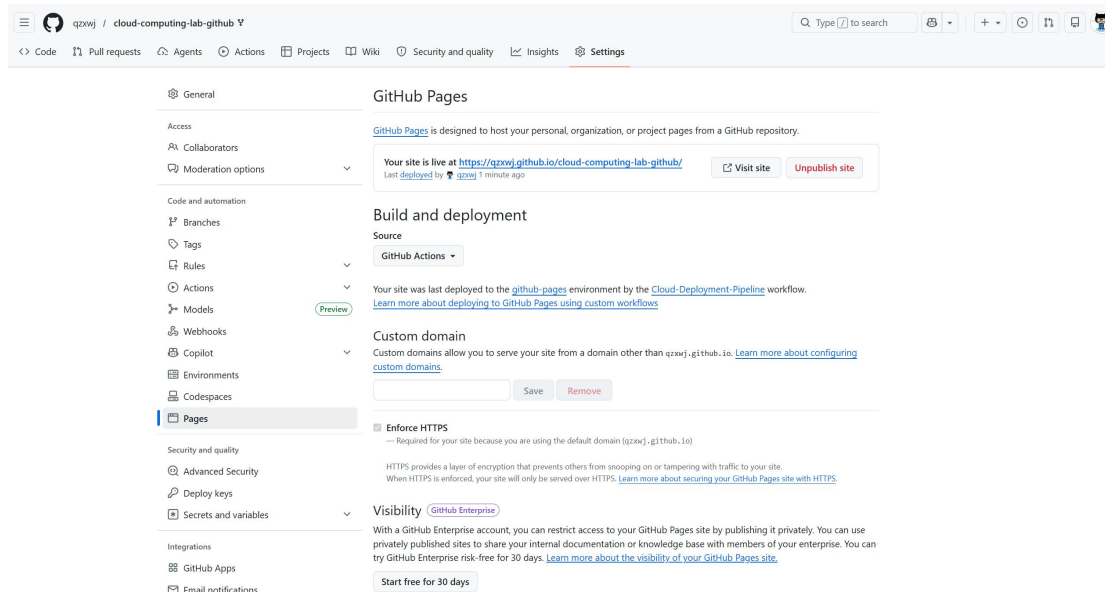
Immediately after committing the change, I opened the Actions tab at the top of the repository page. The Actions tab provides a visual interface for checking workflow

runs, including their status, start time, running jobs, and execution results. This screenshot records the observation stage of the experiment. Instead of guessing whether the deployment has started, I can use the cloud platform's monitoring interface to confirm the workflow behavior. This reflects an important feature of cloud services: users can observe and manage automated processes through dashboards.



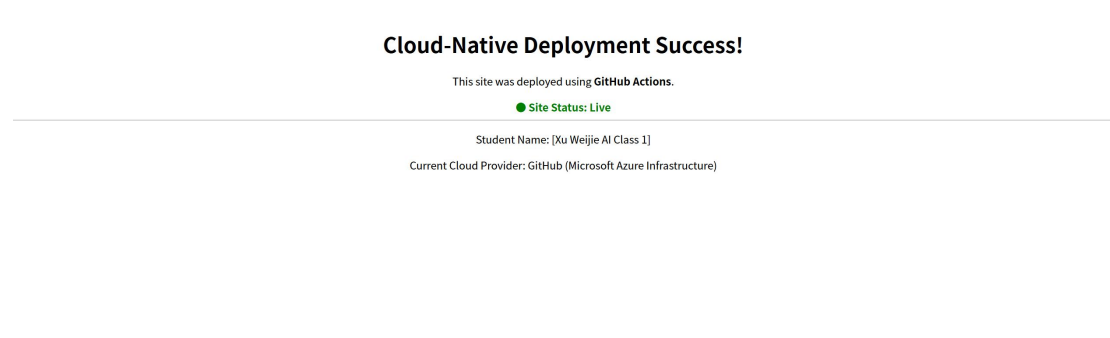
Step 10: Observing the Deployment Workflow

The Cloud-Deployment-Pipeline started running automatically after the commit was submitted. During this process, GitHub allocates an on-demand virtual machine, usually an Ubuntu runner, to execute the workflow jobs. The runner checks out the repository, processes the deployment instructions, and publishes the website through GitHub Pages. This step demonstrates on-demand computing because the computing environment is created only when the workflow needs to run and is released after the job finishes. Users do not need to buy or keep a server running all the time.



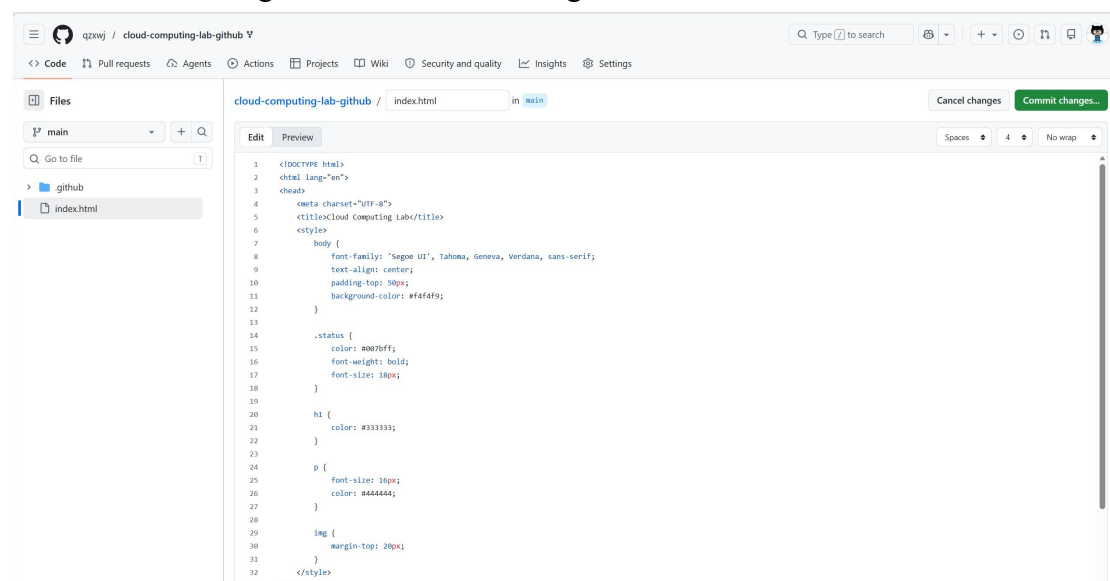
Step 11: Confirming Successful Workflow Completion

When the workflow finished, the green check mark showed that the deployment process completed successfully. This means the automated runner executed the required steps without errors and the website files were successfully published to the GitHub Pages hosting service. This screenshot is important evidence for the experiment because it verifies that the managed pipeline worked as expected. It also shows the reliability advantage of automated deployment: the workflow status clearly tells the user whether the deployment succeeded or whether troubleshooting is needed.



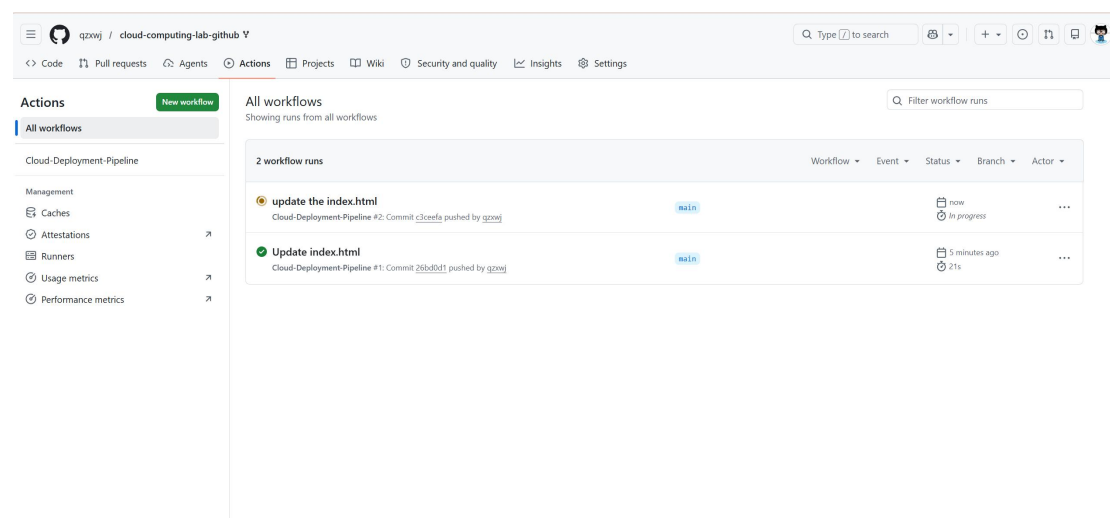
Step 12: Returning to Pages for the Live URL

After the workflow completed successfully, I returned to Settings > Pages to find the live website URL. GitHub Pages provides this public address after the site has been deployed. The URL is the final access point that connects the repository content with external users on the Internet. This step shows how a cloud platform transforms source files into an accessible web service. The student does not need to configure a domain name, install web server software, or manage network routing manually, because GitHub Pages handles these hosting details.



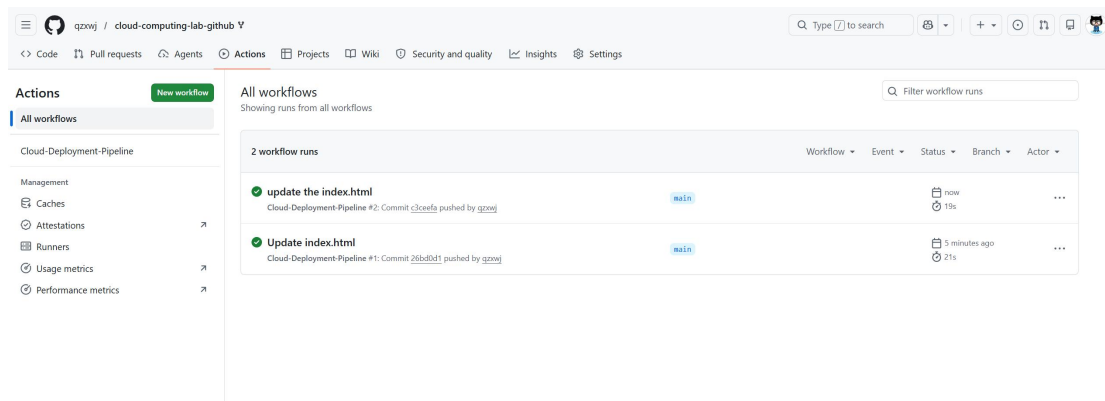
Step 13: Opening the Deployed Website

I opened the live URL on my laptop to verify that the deployed website could be accessed successfully. The page displayed the updated content, which confirms that the modification in index.html was delivered from the repository to the public site. This step demonstrates the Platform as a Service model. GitHub manages the underlying infrastructure, including servers, deployment environment, HTTPS access, and global delivery, while I only focus on editing the website content. The successful browser result proves that the cloud-hosted application is available outside the repository page.



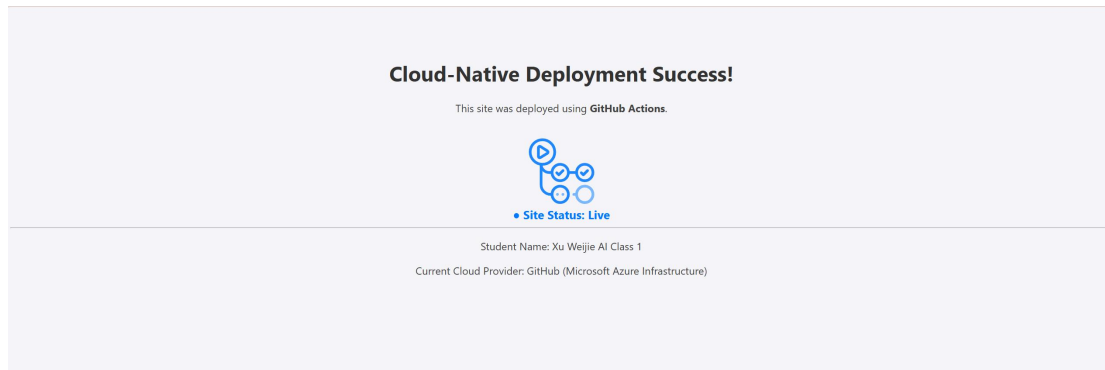
Step 14: Modifying CSS Theme or Page Style

Beyond changing the name, I modified the style settings in the style block, such as the background color, font display, or status text color. CSS changes affect the user interface rather than only the text content, so this step verifies that different types of front-end resources can also be updated through the cloud deployment workflow. After committing the style modification, GitHub Actions can run again and publish the new visual appearance. The cloud concept here is that UI configuration and web presentation files are stored as part of the cloud project and can be updated through the same automated pipeline.



Step 15: Adding an Image and Observing Automatic Redeployment

Finally, I added an `img` tag to include an external image resource in the web page. This step demonstrates multimedia integration and shows that a cloud-hosted static site can reference resources beyond plain HTML text. After the image code was committed, GitHub Actions automatically triggered another deployment workflow, proving that the pipeline responds to each repository update. This is a practical example of continuous deployment. Every committed change, whether it is text, CSS, or an image element, can start a new build and publish the latest version of the website without manual server operation.



四、Reflection

1. What is an EC2 instance essentially?

My website is stored and managed on GitHub's cloud platform. More specifically, the source files of the website, such as `index.html` and related HTML/CSS resources, are stored in my forked GitHub repository. After I enabled GitHub Pages and selected GitHub Actions as the deployment source, GitHub automatically built and deployed the website through its cloud-based workflow. The final website is not stored on my local computer or on a physical server that I manage by myself. Instead, it is hosted by GitHub Pages, which provides a cloud-based web hosting service. When users visit the live URL, they access the deployed version of

the website through GitHub's online infrastructure. This reflects an important feature of cloud computing: users only need to manage their project content, while the cloud platform handles storage, deployment, hosting, and public access.

2. What is the main advantage of server virtualization?

In this experiment, I did not need to install any additional software on my local computer. All the main operations were completed through the web browser, including logging in to GitHub, forking the repository, editing the index.html file, committing changes, enabling GitHub Pages, and checking the GitHub Actions workflow. This shows the convenience of cloud-based development and deployment. In a traditional deployment process, users may need to install a code editor, a web server, deployment tools, or other local software. However, in this experiment, GitHub provided an integrated online environment for code hosting, file editing, automatic building, and website deployment. Therefore, the experiment helped me understand that cloud platforms can reduce local environment requirements and make deployment easier for beginners.

3. Why can cloud computing create servers quickly?

During the experiment, some problems may occur in different stages. For example, after forking the repository, the GitHub Pages option may not immediately show the live website URL, because the deployment workflow needs time to finish. The solution is to open the Actions tab and check whether the Cloud-Deployment-Pipeline is still running or has completed successfully. Another possible problem is that the website content does not update immediately after committing changes. This may be caused by deployment delay or browser cache, so I can wait for a short time, refresh the page, or open the website in a new browser window. In addition, if the workflow fails, I should check whether the index.html file was modified correctly and whether there are syntax errors in the HTML or CSS code. Through these problems and solutions, I learned that cloud deployment is highly automated, but users still need to carefully check configuration settings, workflow status, and source code correctness.

附录 程序命令代码

附录 1: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<title>Cloud Computing Lab</title>
<style>
  body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    text-align: center;
    padding-top: 50px;
    background-color: #f4f4f9;
  }
  .status {
    color: #007bff;
    font-weight: bold;
    font-size: 18px;
  }
  h1 {
    color: #333333;
  }
  p {
    font-size: 16px;
    color: #444444;
  }
  img {
    margin-top: 20px;
  }
</style>
</head>
<body>
  <h1>Cloud-Native Deployment Success!</h1>
  <p>This site was deployed using <b>GitHub Actions</b>.</p>
  
  <div class="status">● Site Status: Live</div>
  <hr>
  <p>Student Name: Xu Weijie AI Class 1</p>
  <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
</body>
</html>
```